SEPOLIA TESTNET • SOURCE-BACKED EVALUATOR GUIDE

AIChainPay Technical Walkthrough

From a new MetaMask wallet to an authenticated in-chat vault payment: architecture, code functions, security checks, recorded evidence, startup steps and viva answers.

Evidence rule. “Implemented” means present in source; “local test” means an automated/local-chain result; “Sepolia verified” requires a receipt/event; “ChatGPT verified” requires real-client and connector evidence. These are not interchangeable.

Prepared from the repository on 20 October 2026. Sepolia only: chain ID **11155111**. Vault `0x915fCDd60d648dC9e9F558e84f805C28269E0aAC`. All amounts in this guide are test ETH.

Public addresses, transaction hashes, source paths and commands only. No private keys, tokens, cookies, signed payloads or secret RPC URLs are included. This is an educational testnet project, not audited financial infrastructure.

Contents

1. Problem, goal and fundamentals
2. Architecture and account model
3. Owner setup, balance read and payment execution
4. Solidity contract: every callable function and check

5. Frontend, backend, OAuth, database and signer functions
6. MCP tools, card approval and payment states
7. Failures, trust boundaries and security
8. Running and demonstrating the system
9. Tests, Sepolia receipts and requirement evidence
10. Presentation, viva questions, glossary and claims to avoid

1. Problem, goal and fundamentals

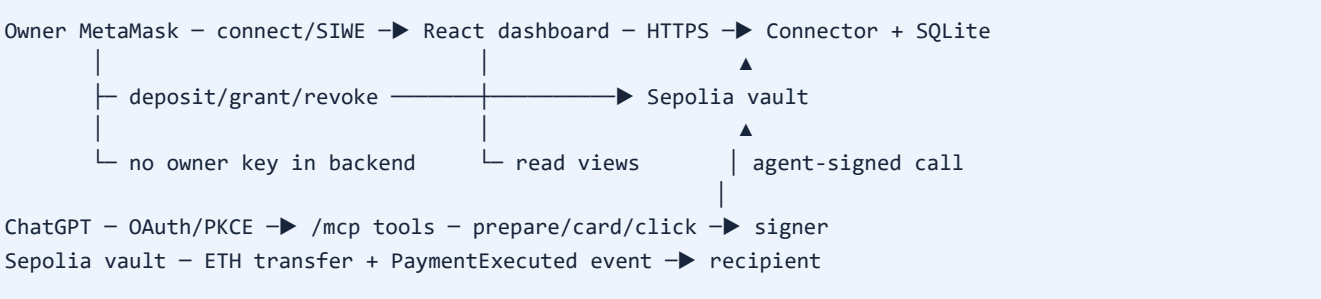
A blockchain transaction normally needs an account's private-key signature. In a conventional DApp, a user revisits the website and approves every payment in MetaMask. An AI chat cannot safely or legitimately possess the owner's MetaMask key. AIChainPay therefore separates *one-time owner authorization* from *routine bounded payment execution*.

The owner deposits small Sepolia test ETH into a Solidity vault and signs a grant for a separate agent address. That grant has an aggregate allowance, per-payment cap, expiry and optional recipient list. The owner can revoke it. A ChatGPT user authenticates as that wallet through SIWE and OAuth, asks MCP tools to prepare a payment, reviews an exact in-chat card and clicks Confirm. The agent signs the resulting vault call and pays gas. The contract, not the AI model, independently enforces the grant. Routine approval does not return to the website.

Terminology: an EOA is a private-key-controlled account; wei is the smallest ETH unit (10^{18} wei = 1 ETH); gas pays transaction computation; a smart contract is public on-chain code; an ABI tells ethers.js how to encode calls and decode events; a receipt proves a transaction was mined, while a prepared preview does not.

> **Not PayBox itself.** This is a PayBox-inspired experience using a Solidity vault and dedicated agent signer. It does not use PayBox custody/MPC infrastructure. It never transfers routine payments directly from the owner's ordinary MetaMask balance.

2. Architecture, identities and multi-user behavior



Owner: signs login message and setup transactions in MetaMask. **Deployer:** signed the one-time contract deployment; it has no privileged user-spending role. **Agent:** one configured backend signer, paid Sepolia

ETH for gas, can spend only a given owner's deposited credit under that owner's on-chain grant. **Recipient:** receives actual ETH from the vault, not from ChatGPT or from the agent wallet.

The deployed contract is **not hard-coded to the original wallet**. In `contracts/AIChainPay.sol`, `balances[msg.sender]` credits each depositor, and `permissions[owner][agent]` separates grants. `connector/src/auth/siwe.ts` accepts any valid supported EOA signature, creates that wallet's session, and `connector/src/payments/prepare.ts` scopes requests to the authenticated owner and OAuth client. A second person can use the same contract and hosted connector with their own MetaMask EOA, but starts with zero vault credit and no grant. They must deposit their own test ETH and authorize the configured agent. A smart-contract wallet is rejected by v1 SIWE login. The Solidity suite includes a positive second-owner deposit/grant/agent-payment case and checks the first owner's credit is unchanged. The second person's full browser/chat workflow was not separately manually demonstrated.

Technology	Why it is used	Live requirement?
Solidity + OpenZeppelin ReentrancyGuard	Public, immutable vault spending checks and reentrancy protection.	Yes, deployed Sepolia bytecode.
ethers.js	MetaMask BrowserProvider, RPC reads, contract calls, agent signing, event decoding.	Yes.
Hardhat	Compilation, local chain, tests and deployment.	Build/deploy tool; local chain not needed for live Sepolia demo.
React + Vite	Owner setup/dashboard; connector serves its built output.	Built UI yes; separate Vite dev server no.
SIWE + OAuth/PKCE	Owner message-signature login versus chat-client API access. Neither is an on-chain grant.	Yes.
MCP + MCP Apps card	ChatGPT reads, preparation, exact in-chat Confirm/Reject and status.	Yes for chat flow; Claude unverified.
SQLite	Sessions, OAuth, immutable requests, one-use card capabilities, nonce journal, audit and event index.	Yes; persistent private data.
HTTPS/ngrok	Gives cloud ChatGPT a reachable local /mcp and matching OAuth origin.	Public HTTPS required; ngrok is replaceable with stable hosting.

3. Exact end-to-end journeys

3.1 Owner setup

Step and code	Input and output	Signer / writes
`connect()` in `frontend/src/App.tsx`	`eth_requestAccounts` → selected public wallet and chain.	No chain transaction or SQLite write.
`login()` → `POST /api/auth/challenge` → `createChallenge()`	Owner address → SIWE message, one-use nonce, five-minute expiry.	SQLite challenge write; no transaction.

`signMessage()` → `POST /api/auth/verify` → `verifyLogin()`	Exact message and wallet signature → HttpOnly browser session, CSRF token.	Owner signs a message; SQLite consumes nonce and stores hashed session. No ETH moves.
`deposit()` → contract `deposit()`	Positive ETH value → owner vault credit and `Deposited` event.	Owner MetaMask transaction on Sepolia; no agent authority implied.
`grant()` → contract `grantAgent()`	Agent, total, maximum, expiry, optional addresses → new permission epoch and event.	Owner MetaMask transaction on Sepolia. Blank list permits any otherwise valid recipient.

3.2 Chat balance read

ChatGPT connects to the public HTTPS `/mcp` via OAuth. The connector's `createOAuthProvider()` handles authorization code + PKCE and wallet-linked consent; `verifyBearer()` checks token expiry, audience and scope on every MCP request. `createMcpServer()` binds tools to the token's owner. `get_contract_balance` calls `ChainService.balance(owner)`, which reads contract `getBalance(owner)`. A JSON result returns wei/ETH. The connector writes an `mcp_request` audit row in SQLite; the balance itself is read from Sepolia and no transaction is sent. `get_native_balance` is different: it reads ordinary wallet ETH.

3.3 Chat payment

Step and code	Input → output	Signer / state
`prepare_payment` → `preparePayment()`	Recipient, decimal amount, memo, client request ID → UUID, bytes32 ID, digest, deadline and exact preview.	No signer/transaction. Reads Sepolia permission/balance; inserts immutable SQLite request.
`show_payment_confirmation` → `issueChatApproval()`	UUID + digest → in-chat UI resource and private random capability.	No ETH. SQLite stores only capability hash, owner/client/digest/mode/expiry.
Card click → `confirm_payment_card` → `decideChatApproval()`	Exact UUID/digest, capability, approve/reject → one-use decision.	SQLite decision/audit. Reject or verify-only mode broadcasts nothing.
`ExecutionService.execute()` and `signAndBroadcast()`	Live decision → preflight, encoded call, signed transaction/hash.	Agent signs; SQLite CAS, nonce and encrypted signed bytes persisted before Sepolia broadcast.
Contract `executeAgentPayment()`	Owner/recipient/amount/memo/request ID/epoch/deadline → recipient ETH and event.	Agent pays gas; vault's owner credit and allowance decrease on Sepolia.
`get_request_status` → `status()/reconcile()`	UUID → persisted state, receipt hash, confirmation count, explorer URL.	No new payment; Sepolia receipt read and SQLite reconciliation.

4. Solidity contract function-by-function

Source: `contracts/AIChainPay.sol`; exported interface: `packages/shared/src/AIChainPay.abi.json` and `packages/shared/src/index.ts`. The actual ABI names below are the deployed design.

Function / state	Plain-language behavior and checks

`receive()`, `fallback()`	Reject an ordinary direct ETH send or unknown call. ETH can still be forcibly sent at EVM level; forced surplus does not create a user's credit.
`deposit()`	Payable, requires nonzero `msg.value`; credits `balances[msg.sender]`, increases `totalLiabilities`, emits `Deposited`. No special owner address exists.
`withdraw(amount)`	Owner withdraws their own credited ETH, nonzero and not beyond balance. Decrements accounting before transfer; reverts on failed transfer; protected by `nonReentrant`.
`getBalance(owner)` / public `balances(owner)`	Read per-owner vault credit; anyone can read public chain state, but only the owner controls withdrawal/grant.
`grantAgent(agent,totalAllowance,maxPerPayment,expiresAt,recipients)`	Caller grants their own funds only. Agent cannot be zero/vault/self; total and max positive, max ≤ total; expiry must be future; ≤20 distinct valid recipients. Replaces grant, increments epoch, emits `AgentGranted`. Empty list means unrestricted recipients.
`revokeAgent(agent)`	Caller invalidates their grant to that agent: advances epoch, sets inactive/zero limits and emits `AgentRevoked`. A later grant is possible; old requests with old epoch remain stale.
`getAgentPermission(owner,agent)`	Read active flag, remaining allowance, maximum, expiry, epoch and recipient policy.
`isAllowedRecipient(owner,agent,recipient)`	Read whether current epoch's list permits recipient; if unrestricted, returns true. Other payment checks still apply.
`pay(recipient,amount,memo,requestId)`	Optional direct owner-signed payment from their vault credit. Not the AI path. Calls the shared validation/execution code and is non-reentrant.
`executeAgentPayment(owner,recipient,amount,memo,requestId,permissionEpoch,requestDeadline)`	Agent-only delegated route: active grant for `(owner,msg.sender)`, current time before grant expiry, matching epoch, request not past deadline, valid payment, amount within max and remaining allowance, recipient allowed. Decrements allowance, then executes; non-reentrant.
`_validatePayment(...)`	Requires positive amount; recipient not zero, vault or owner; nonzero unused bytes32 request ID; memo ≤140 UTF-8 bytes; enough owner vault credit.
`_execute(...)`	Deducts owner credit/liabilities, marks request ID used, stores payment/participant indexes, calls recipient with exact ETH, reverts if transfer fails, emits `PaymentExecuted`. A revert rolls all preceding changes back.
`getPaymentCount(participant)`, `getPayment(participant,index)`	Read recorded payments involving an owner/recipient; out-of-range index reverts. Not full wallet history.

Public getters: `totalLiabilities()`, `usedRequestIds(owner,id)`, `MAX_MEMO_BYTES()` and `MAX_ALLOWED_RECIPIENTS()` are compiler-generated for public state/constants. Event and account data are public on Sepolia; do not put secrets in memos.

5. Application function catalogue

This catalogue covers production runtime functions and helpers, not test-fixture methods or third-party library internals. A file path plus function name identifies the exact source. The application uses `ethers.js` v6, TypeScript, React and Express.

5.1 Shared validation — `packages/shared/src/index.ts`

Function or constant	Purpose
`SEPOLIA_CHAIN_ID`, `LOCAL_CHAIN_ID`, `EXPLORER_BASE_URL`	Define accepted live/test chain identifiers and public explorer base.
`VAULT_ABI`	Human-readable interface consumed by frontend and connector; a full JSON ABI is exported separately.
`parseAmountEth(value)`	Accepts plain positive decimal ETH with at most 18 fractional digits; calls ethers `parseEther`; rejects scientific notation, zero and malformed input.
`normalizeAddress(value)`	Requires exact 20-byte hexadecimal address, applies checksum with ethers `getAddress`; ENS is unsupported.
`validateMemo(value)`	Enforces 140 UTF-8 bytes, matching the contract (not merely 140 characters).

5.2 Dashboard — `frontend/src/App.tsx`

Function / state	Purpose
`eth()`, `provider()`	Locate injected MetaMask provider, wrap it in ethers `BrowserProvider`.
`api(path,init)`	Credentialed JSON HTTP wrapper; propagates backend error messages.
`short()`, `displayEth()`, `formatDate()`, `errorText()`, `Address()`	Formatting and address copy/explorer display; shortened addresses are display-only, not transaction inputs.
`refreshWallet()`	Reads selected account/chain without prompting; clears wallet state when MetaMask absent.
`refreshData()`	Only when SIWE session matches selected wallet: fetches owner-scoped dashboard, vault events and prepared requests; clears these on mismatch.
React effects	Load public config and current session; subscribe to account/chain changes; refresh data; resume same-origin OAuth interaction return.
`act(label,fn)`	Common busy/error/notice wrapper for UI operations, then refreshes data.
`connect()`, `switchNetwork()`	Request MetaMask account; switch to configured chain. Automatic network-add fallback is local-chain-only.

`signer()`	Reject wrong chain or changed account; returns selected MetaMask signer. The connector never receives this key.
`login()`	Get SIWE challenge, sign message in MetaMask, verify with backend; no blockchain transaction.
`ownerTx(label,fn)`	Construct ethers vault contract with owner signer, send selected transaction, wait for receipt.
`deposit()`, `withdraw()`	Strict ETH conversion; owner-signed vault credit/withdrawal transactions.
`grant()`	Parse total/cap, require cap ≤ total, integer expiry 1–720 hours, at most 20 distinct optional recipients, then call `grantAgent`.
`revoke()`, `directPay()`	Owner-signed grant revocation; optional direct owner vault transfer with fresh random bytes32 ID. Direct pay is not chat execution.
`logout()`	Clears browser session via CSRF-protected API; explicitly does not revoke chain grant.
`vaultReady`, `chatActivityObserved`, `completedSetupSteps`	Four-step display now derives from this wallet's connection, SIWE session, funded unexpired grant and owner-scoped authenticated MCP-request record. Activity means “observed historically,” not “token currently valid.”

5.3 Connector startup, routes and chain reads

Code location / function	Purpose and writes
`connector/src/config.ts:loadConfig()`, `required()`	Validate Sepolia/default chain, HTTPS issuer/origin, RPC, bytecode hash, adapter gating, secret length, TTL and agent key. Local 31337 requires explicit test flag.
`connector/src/db/index.ts:openDatabase()`, `migrate()`	Open SQLite with WAL, foreign keys, busy timeout; create/migrate tables. This modifies only local DB.
`connector/src/blockchain/index.ts:ChainService()`, `check()`	Create RPC provider/vault interface; verify chain ID, nonempty code, memo identity and expected code hash before service/execution.
`ChainService.balance()`, `permission()`, `allowed()`	Read owner credit, owner-agent grant and current recipient policy from Sepolia; no chain writes.
`connector/src/blockchain/indexer.ts:PaymentIndexer.sync()`, `syncNow()`	Read `PaymentExecuted` logs from deployment block in bounded 1,000-block ranges, rescan a 12-block overlap for shallow reorgs, store events/cursor in SQLite.
`PaymentIndexer.recentFor()`	Return owner/recipient-scoped vault events with cursor and explorer URL; not all wallet transactions.
`connector/src/index.ts:main()`	Assemble config, DB, chain, indexer, signer service, OAuth and MCP server; validate chain, recover pending signed payments in live mode, serve built frontend on port 3000.

`cookie()`, `session()`, `originGuard()`, `csrfGuard()`, `escapeHtml()`	Parse session cookie, load hashed session, reject foreign browser origins/CSRF, escape OAuth interaction HTML. Diagnostic logging redacts sensitive redirect path IDs/query values.
`GET /health`, `GET /api/config`	Read-only readiness and public network/vault/MCP configuration. Never expose keys.
`POST /api/auth/challenge`, `/verify`, `GET /api/auth/me`, `POST /logout`	Create/consume SIWE challenge, set secure HttpOnly cookie on HTTPS, show or end browser session. Challenge/session/audit are SQLite writes; no Sepolia transaction.
`GET /api/dashboard`	Requires wallet session; reads wallet/vault/agent/grant from chain, recent audit and whether *any* authenticated MCP request exists for this owner.
`GET /api/payments`, `/api/requests`	Owner-scoped indexed vault events and prepared request states; requests endpoint marks expired unsigned intents in SQLite.
`/.well-known/oauth-protected-resource*`, `ALL /mcp`	Advertise OAuth resource; verify bearer on each MCP request, record owner audit, run stateless streaming-HTTP transport with owner-bound tools.
`GET/POST /interaction/:uid`	Wallet-linked OAuth login/consent page, same-origin/CSRF checks and allowed scopes; this is connection authorization, not vault permission.

5.4 Authentication and persistence

Function / file	Purpose
`hashSession()`, `isEoaAccountCode()` — `connector/src/auth/siwe.ts`	Hash browser cookie before storage; permit empty EOA code or EIP-7702 EOA delegation indicator, not arbitrary contract-wallet code.
`createChallenge()`, `verifyLogin()`, `getBrowserSession()` — same file	Issue one-use SIWE message bound to domain/URI/chain/time; verify EOA signature and nonce atomically; return/lookup 12-hour browser session.
`signingKey()`, `sqliteAdapter()` — `connector/src/auth/oauth.ts`	Persist local OAuth signing JWK privately; map OAuth objects, expiry, consumption and revocation to SQLite.
`createOAuthProvider()`, `verifyBearer()` — same file	Configure OAuth authorization code + PKCE, ChatGPT client metadata, /mcp audience and one-hour token; verify bearer expiry/audience/scope and return owner/client principal.
`users`, `challenges`, `browser_sessions`, `oauth_objects`	Identity and client connection records in SQLite. Browser cookie is stored only as hash.
`payment_requests`, `chat_approvals`, `transaction_attempts`	Immutable intent/state, hashed one-use card tokens, unique nonce/hash and encrypted signed transaction bytes.
`audit_entries`, `index_cursors`, `payment_events`	Authenticated activity, event-sync progress and on-chain event cache.

6. MCP tools, confirmation and state machine

All tools are registered in `connector/src/mcp/server.ts:createMcpServer()`. ChatGPT receives a wallet-specific principal from `verifyBearer()`; the model cannot choose a different owner or network through the tools.

MCP tool	Meaning / on-chain effect
`get_wallet_address`, `get_supported_network`	Return authenticated EOA and Sepolia chain; no transaction.
`get_native_balance`, `get_contract_balance`, `get_agent_permission`	Read ordinary wallet ETH, owner vault credit and owner-specific grant; no transaction.
`prepare_payment`	Validate and persist an off-chain intent. It does not pay.
`show_payment_confirmation`	Provide exact preview and MCP Apps HTML card, mint a private one-use capability. It does not pay.
`confirm_payment_card`	App-only Confirm/Reject callback. Verify-mode Confirm records choice/no ETH; Reject sends no ETH; live Confirm can invoke signer.
`execute_payment`	Legacy model-call tool. In active `chatgpt_app` mode it refuses execution and directs user to the card. A remembered tool approval is not payment approval.
`cancel_payment_request`	Cancel owner's own still-prepared intent; SQLite only.
`get_request_status`, `get_transaction`, `get_recent_transactions`	Poll request/receipt, look up a chain transaction, or list scoped vault events. None creates a payment.
`probe_write_confirmation`	Historical harmless audit-only tool used to inspect generic client write prompts. It never authorizes a payment.

6.1 Preparation and card function reference

Function / file	Exact responsibility
`requestForPrincipal()`, `preview()`, `assertExecutionDetails()` — `connector/src/payments/prepare.ts`	Scope UUID to owner+OAuth client; make the human-readable immutable preview; compare every legacy execution field to that saved preview.
`preparePayment()`, `cancelPayment()` — same file	Strict address/amount/memo/client ID, idempotency key and conflict check, live grant/balance/recipient check, random bytes32 request ID, ABI-encoded digest, bounded deadline, SQLite insert; cancellation only from prepared state.
`requireChat()`, `currentRequest()` — `connector/src/payments/chat-approval.ts`	Require ChatGPT client ID plus execute scope and the same owner/client/digest; require prepared and unexpired.
`issueChatApproval()`, `decideChatApproval()` — same file	Generate 32-byte random capability, store hash/mode and supersede prior cards; atomically verify token, owner, client, intent, mode and expiry; audit decision/reject request if applicable.
`chatAppConfirmation()` — same file	Live adapter consumes exactly one fresh approved execute-mode capability for the same owner/client/digest; otherwise refuses.

`PAYMENT_CARD_HTML` — `connector/src/mcp/payment-card.ts`	Renders full amount, recipient, owner, vault, agent, network, memo, deadline, epoch, UUID and digest. Uses text-safe rendering and trusted user-event guard; calls app-only tool, never auto-confirms on render.

6.2 Signer and recovery function reference — `connector/src/payments/execute.ts`

Function	Responsibility
`cipherKey()`, `encryptSigned()`, `decryptSigned()`	Derive a key from persistent session secret; AES-256-GCM encrypt/decrypt raw signed transaction in private SQLite. Changing the secret can break recovery.
`ExecutionService()`, `serialized()`	Create wallet only with configured agent key; serialize signing in one process. Deployment assumes one writer/agent nonce stream.
`execute()`	Check execute scope, owner/client/digest, state and deadline; SQLite compare-and-swap from prepared to confirming; consume confirmation; move to signing or reject before any broadcast.
`signAndBroadcast()`	Recheck chain/grant/epoch/cap/balance/recipient; static-call simulation, gas estimate and gas balance; choose unique nonce; sign type-2 transaction to vault with zero outer ETH value; persist encrypted bytes/hash/nonce before sending.
`broadcastStored()`	Broadcast the identical signed bytes. RPC timeout or already-known response is uncertain, not permission to sign a new payment.
`status()`, `reconcile()`, `setState()`	Poll and update state. Reconciliation checks receipt, chain ID, contract, agent, nonce, exact calldata, zero outer value, matching `PaymentExecuted` fields, block hash and required confirmations; mismatch becomes `reconciliation_required`.
`recover()`	At live startup, release expired unsigned claims, expire old intents, reconcile known hashes and rebroadcast identical durable signed bytes if needed.
`chatGptNativeConfirmation()`	Legacy host-tool-approval trust adapter; not the active card-based payment route. Its own client prompt is not cryptographic proof of a human.

```
prepared → confirming → signing → signed → submitted → confirmed
             ↘ rejected      ↘ preflight failure
prepared → cancelled or expired
submitted/confirmed → reverted or reconciliation_required on chain mismatch/reorg
```

A submitted hash is not a confirmed payment. `get_request_status` must poll; the project does not push unsolicited completion to ChatGPT. A successful HTTP tool response or an AI-written sentence alone is not a receipt.

7. Safety, failure modes and trust limits

- **Two separate permissions:** OAuth authorizes ChatGPT to call an owner's tools; the Solidity grant authorizes the agent to spend this owner's vault credit. Browser logout or plugin disconnection does not revoke on-chain permission.
- **One-use exact confirmation:** the card capability is private UI metadata and stored only hashed. The server binds it to request/digest, owner, OAuth client, mode and deadline. Verify-only approvals cannot become live approvals. The model cannot supply a simple `approved:true`.
- **Human-proof limit:** the host must isolate private metadata and enforce app-only callback visibility. The component's `event.isTrusted` is not server-verifiable human attestation. A malicious host/backend or stolen full OAuth access may defeat this layer; on-chain allowance/cap/expiry/revocation limit damage.
- **Crash/nonce safety:** durable signed bytes and unique nonce prevent a restart from creating another payment for the same request. Do not use the agent key for unrelated transactions while the connector runs. A mismatched/reorganized receipt needs investigation, not blind retry.
- **Public data:** recipient, amount, memo and events are public on Sepolia. Do not put secrets in memo. Use only small test funds.
- **Revocation:** once mined, it blocks future delegated execution under that grant. It cannot undo completed transfers and cannot guarantee ordering ahead of an already-pending agent transaction. A later grant creates a new epoch.
- **Availability:** if backend or tunnel stops, chat tools stop; already-deployed contract and credited ETH remain on Sepolia. A stable HTTPS host can replace ngrok. Local SQLite and agent key must be protected and backed up; copying only Git files to a new machine does not transfer existing OAuth sessions or grant authority to a different agent.

8. Running and demonstrating AIChainPay

8.1 Existing validated Windows workspace

Prerequisites: Node.js ≥22.13, npm, this repository with its existing ignored local RPC/connector configuration, assigned ngrok HTTPS hostname and sufficient agent gas. Do not start a second connector on port 3000. The currently deployed contract is reused; no deployment transaction is needed. Run these in PowerShell from the repository:

```
cd D:\AIChainPay
npm.cmd ci
npm.cmd run build
npm.cmd run verify:sepolia
npm.cmd run start:sepolia -- --live
```

The explicit live switch works only for the already verified local connector: `scripts/sepolia-connector.ts` requires locally recorded real-host verification approval and rejection, matching agent key and live adapter settings. A new installation must first use `npm.cmd run start:sepolia`, which forces execution disabled. Never set verification flags merely to bypass the gate.

In another PowerShell window, only if the tunnel is offline:

```
cd D:\AIChainPay
.\scripts\start-stable-ngrok.ps1 -PublicBaseUrl https://upscale-nanometer-duvet.ngrok-free.dev -
AllowLiveConnector
```

Enter ngrok authentication only in its hidden local prompt. Keep connector and tunnel windows open. Check `https://upscale-nanometer-duvet.ngrok-free.dev/health`; the dashboard is the same origin and ChatGPT's OAuth MCP URL is `https://upscale-nanometer-duvet.ngrok-free.dev/mcp`. An unauthenticated /mcp request should return a bearer challenge, not payment data. This assigned hostname belongs to the operator's ngrok account; it is not guaranteed on another device.

8.2 Fresh operator machine versus a second end user

Fresh operator machine: build from the lockfile; place a Sepolia RPC URL in a private local root `.env` based on `.env.example`; verify deployed manifest/bytecode with `npm.cmd run verify:sepolia`; acquire a stable public HTTPS origin; run `npm.cmd run setup:sepolia-connector -- https://YOUR-HTTPS-ORIGIN` once. This makes a *new* agent signer and local SQLite/OAuth state without broadcasting. Fund that new agent with small Sepolia test ETH for gas and have each owner grant that agent. Start disabled; only after genuine card verification can the guarded live route be enabled. Never copy/publish private keys, SQLite, signing JWK or tokens. A deployer private key is not necessary to reuse this contract.

Second end user: does not run the backend if the operator hosts it. They open public dashboard, connect a MetaMask EOA on Sepolia, sign the SIWE login message, deposit their own test ETH, grant the displayed agent with total allowance/cap/expiry (optional allowlist), then OAuth-connect their own ChatGPT plugin to the same /mcp endpoint. Their old ordinary wallet ETH remains outside the vault. Without deposit/grant, they can read a zero balance but cannot pay. Another person's grant cannot spend their vault credit.

8.3 Five-minute presentation script

1. **0:00–0:45:** “Normal DApps require returning to a website and signing each payment. We wanted a natural-language chat payment without handing MetaMask's key to an AI.”
2. **0:45–1:45:** Draw owner/agent/vault. “The owner deposits Sepolia ETH and signs bounded permission. The separate agent pays gas; Solidity enforces total, per-payment, expiry and recipient policy.”
3. **1:45–2:45:** “SIWE proves wallet control to dashboard; OAuth connects the chat client; MCP exposes read, prepare, card and status tools. These do not replace the on-chain grant.”
4. **2:45–3:45:** “A fresh in-chat card displays exact payment details. The one-use callback lets the agent sign; the contract checks permission and replay ID, then sends ETH to recipient.”
5. **3:45–5:00:** Show actual Sepolia receipt/event and distinguish verified payment from verify-only screenshot. Mention testnet, host trust, no audit/permanent hosting/GitHub CI claim.

8.4 Ten-minute live demo, with recorded-evidence fallback

1. **0–2 min:** Check /health, Sepolia chain ID and vault address; dashboard shows wallet, vault and grant. Explain setup checklist is owner-specific and authenticated MCP activity is historical evidence, not a live-token monitor.
2. **2–4 min:** In ChatGPT select installed AIChainPay plugin. Prompt: “Use AIChainPay only: call get_supported_network, get_wallet_address, get_contract_balance and get_agent_permission. Do not prepare or execute.” Expect chain 11155111, the selected owner, their vault credit and current limits.
3. **4–6 min:** Prompt: “Use AIChainPay get_request_status for request 7c04c304-d6fa-469f-baf7-4ebd8dcaaba5. Do not prepare or execute another payment.” This request is scoped to the original owner's OAuth client; a second user's plugin cannot read it. For the original owner expect confirmed and the known receipt hash.
4. **6–9 min (optional new transfer):** Only if evaluator authorizes a test payment and pre-demo grant/balance/gas checks pass, use a fresh client request ID, ≤0.0001 ETH, and a recipient you control. Ask ChatGPT to call prepare_payment then show_payment_confirmation with returned UUID/digest, and stop for your click. Read exact live amount/address/network/source before Confirm. The click authorizes that one vault transfer, not unlimited wallet access. Poll get_request_status until confirmed; do not click again if UI spins.
5. **9–10 min:** Explain revoke and show past revoke/regrant receipts; do not send another revoke transaction casually during a short demo. If tunnel/client is unavailable, show the recorded first Sepolia receipt, confirm-only screenshot (labelled no transfer), and test output rather than claim a fresh live demo.

9. Evidence, tests and requirements

Deployment: vault `0x915fCDd60d648dC9e9F558e84f805C28269E0aAC`, chain 11155111, deployment block 11947272, transaction `0xcd059159f582a2783da6f7652d4fec4c390e1696db1b596b2b291ee78005d46d`. Public manifest: `packages/shared/src/deployment.11155111.json`. Runtime code hash/ABI/creation receipt matched in the read-only verifier; Etherscan source publication is not claimed.

Observed payment: original owner `0x0432821e68A84b2F2e8dCD6e7973d14Db1A939df`; agent `0x3d111235700B9C3BEdb097467a3bf98bB7FAfA3F`; recipient `0x94421cF16aAefb1b29B2e9019f7aEF05B83d659f`; 0.0001

ETH. Transaction `0x2bfe8bfc442c084cc012fed2b345044819bad1a11dfef682fea7a7e0cbff01cf` mined in block 11952073 with matching event, owner debit/allowance decrease, exact recipient balance rise and authenticated confirmed MCP status. The verify-only screenshot `docs/screenshots/chatgpt-card-verify-approved.png` did not itself transfer ETH; there is no separate live-card screenshot.

Later controls: owner revoke `0x1e8d94f28b434eb393d8e16e830bdd52cb28b7ccc6ceeddd9fcbf7d72` (block 11955494) and regrant `0x5759f2a59d21ad4e1842cb3c07eff8a1e205498d7ebd08bc8c908612e93e4526` (block 11955598). A read-only simulation during revoked state returned `PermissionInactive`; there was no reverted broadcast. At last check, epoch 3 grant allowed 0.001 ETH aggregate, 0.0001 ETH per payment, any valid recipient, expiry 21 October 2026 13:29:05 UTC. Read again before use. A separate 0.000232 ETH agent transaction `0xce0880e7d03f79a69e190db0b25bcd64fe367b780f3f572bb110006cba20ac62` succeeded on-chain, but its SQLite row remained submitted and independent client-click screenshot is absent; do not present it as a second fully documented chat flow.

Assignment/build goal	Source/test/evidence	Status
Solidity vault, owner/agent, deposit/withdraw, caps/allowlist/expiry/replay, multi-user isolation	`contracts/AIChainPay.sol`; 9 contract tests in `test/contracts/AIChainPay.ts`	Implemented/local tested; deployed Sepolia
Sepolia deployment, ABI/address/config checks	`scripts/deploy.ts`, `scripts/verify-sepolia.ts`, manifest and deployment receipt	Read-only verifier passed; explorer source verification not claimed
MetaMask dashboard setup, direct owner controls	`frontend/src/App.tsx`, deposit/grant/revoke/regrant receipts	Original owner observed; a second real wallet not manually demonstrated
SIWE, OAuth, MCP reads and user isolation	`connector/src/auth/*`, `connector/src/mcp/server.ts`, 7 backend tests	Local tests pass; original owner's ChatGPT reads observed
Immutable preview and exact in-chat confirmation	`prepare.ts`, `chat-approval.ts`, `payment-card.ts`, 5 approval tests; real verify-only screenshot and live audit	Implemented; real card approval observed with host-trust caveat
Agent signing, nonce/crash recovery, receipt reconciliation	`execute.ts`; local execution tests; first live payment receipt	Local negative/recovery tests pass; one full Sepolia chat payment verified
Rejection, over-cap, revoke	Verify-mode rejection audit, over-cap prepare refusal, contract/local tests, revoke/regrant receipts	Different evidence levels; no live over-cap/rejection transaction broadcast
Documentation, screenshots, clean install, GitHub submission	README, this guide, real unconnected Sepolia dashboard and verify-only card screenshots; local clean-install and safety checks	Docs/screenshots exist; no connected-wallet/live-card screenshot, GitHub remote or hosted CI verified

Recorded local results: `npm.cmd test` passed 9 Solidity and 12 connector tests; full build passed with a non-fatal large frontend chunk warning. `test:payment-card` passed in a synthetic browser host; local-card execution checked actual local-chain transfer, rejection, cap, revocation and replay; local execution tests checked concurrent nonces and recovery. These are not substitutes for ChatGPT/Sepolia evidence. A fresh source-only copy installed, built and passed all 21 tests. Tests should be rerun after source changes; no test result should be invented.

10. Fifteen viva questions and answers

1. **Why a vault instead of paying directly from MetaMask?** An AI backend cannot sign as the owner's wallet without taking its private key. The owner deposits only test funds and delegates bounded contract authority.
2. **Why does the deployer need a private key?** Deployment is a transaction that must be signed and funded. The deployed contract has no deployer-only spending role.
3. **Why does the agent need a private key?** It signs `executeAgentPayment` and pays gas. Its key cannot call `withdraw` for someone else's credit.
4. **Why is the owner key only in MetaMask?** Owner deposit, grant, revoke and withdrawal require that person's consent; the backend does not import their key.
5. **Can my evaluator use another wallet?** Yes, a supported EOA can create its own credit/grant on the same contract. It must do its own SIWE login, deposit and grant. A contract-account wallet is not supported by current login.
6. **What does SIWE do?** A short-lived signed message authenticates control of a wallet to the dashboard; it is not an ETH transaction or agent grant.
7. **How is OAuth different?** OAuth gives the chat client scoped API access for that wallet. The Solidity grant independently governs actual vault spending.
8. **Why MCP?** It gives ChatGPT named read, prepare, card and status tools over a standardized remote connection.
9. **Why ngrok?** The local connector needs public HTTPS for cloud ChatGPT and OAuth. Ngrok forwards to this laptop; stable hosting can replace it. It does not hold vault ETH.
10. **What happens if the backend stops?** Chat reads/payments stop, but contract credit and past transactions remain on Sepolia. Owner can later restart or use another compatible interface.
11. **What prevents over-limit payments?** Contract checks credit, remaining allowance, per-payment cap, expiry, epoch and optional recipient list on every delegated transaction.
12. **What prevents duplicates?** Prepared idempotency key, one-use card token, SQLite state/nonce journal and on-chain used request ID. Identical signed bytes can be rebroadcast after uncertainty without a new payment.
13. **How does revocation work?** Owner signs `revokeAgent`; after mining, grant is inactive and epoch advances. A later grant creates a new epoch; completed payments cannot be reversed.
14. **Does the contract know a human clicked?** No. The server trusts ChatGPT's UI isolation and one-use callback; Solidity knows only agent signature and on-chain limits.
15. **What proves payment success?** Mined receipt with expected sender/vault/calldata, matching `PaymentExecuted` event, recipient ETH increase and reconciled confirmed status—not a prepared preview or AI statement.

11. Glossary and unsupported claims

Term	Meaning	Term	Meaning

EOA	Private-key-controlled wallet	Vault	Contract holding credited ETH
Sepolia	Ethereum test network	Gas	Transaction computation fee
Wei	10^{-18} ETH	RPC	Endpoint for chain reads/broadcast
ABI	Contract call/event interface	Event	Public contract log in receipt
Grant	Owner's bounded agent authority	Epoch	Grant version invalidating stale requests
SIWE	Wallet-signed website login message	OAuth/PKCE	Scoped chat-client authorization flow
MCP	Protocol exposing chat tools/resources	Intent digest	Hash binding payment terms; not a signature
Nonce	Agent transaction sequence number	SQLite	Local durable connector state
Receipt	Mined transaction result	ngrok	HTTPS route to local connector

Do not claim: mainnet or production readiness; audit; PayBox-equivalent MPC/custody; routine transfer from the owner's ordinary wallet; cryptographic proof of a human click; automatic payment from a remembered tool permission; Claude support; permanent hosting; Etherscan source publication; a real live-card screenshot when only the verify-only screenshot exists; all browser flows tested with a second owner; the 0.000232 ETH request fully reconciled; or GitHub publication/CI without an actual remote and run.

Source of truth: `contracts/AIChainPay.sol`, `packages/shared/src/AIChainPay.abi.json`, `frontend/src/App.tsx`, `connector/src/**`, deployment manifest, test suite, public Sepolia receipts and `IMPLEMENTATION_STATUS.md`. Historical status entries are snapshots; read the latest audit first. No private configuration is reproduced in this guide.